Les Conteneurs sous Linux (2)

-Conteneur pdf converter-

🎯 Objectifs :

la création d'un convertisseur automatique de fichiers texte vers PDF dans un conteneur.

À l'issue de ce TP, l'étudiant sera capable de :

- Créer et configurer un environnement Linux adapté à l'exécution de conteneurs rootless (sans privilèges root).
- Préparer un système de **stockage persistant** pour permettre l'échange de fichiers entre l'hôte et le conteneur.
- Utiliser des **Dockerfiles** pour construire des images personnalisées.
- Lancer des conteneurs Podman avec des volumes partagés et des noms explicites.
- Tester le bon fonctionnement d'un conteneur à travers une conversion automatisée de fichiers.
- Générer et configurer un **service systemd** pour automatiser l'exécution du conteneur au démarrage.
- Renforcer la compréhension du cycle de vie d'un conteneur : construction, exécution, test, persistance et automatisation.

Exercice 2 : Création d'un convertisseur texte ➜ PDF avec Podman

---

☑ 1. Créer un utilisateur nommé `pod` avec le mot de passe `pod`

```
sudo useradd pod
echo "pod:pod" | sudo chpasswd
```

```
[ons@mylocalhost ~]$ sudo useradd pod
[sudo] password for ons:
[ons@mylocalhost ~]$ echo "pod:pod" | sudo chpasswd
[ons@mylocalhost ~]$
```

🔍 Explication :

- `useradd pod` : crée un nouvel utilisateur nommé `pod`.
- `chpasswd` : permet d'automatiser la définition du mot de passe (`pod`) pour ce compte.

☑ 2. Créer les répertoires `/data/input` et `/data/output`

```
sudo mkdir -p /data/input /data/output
```

```
[ons@mylocalhost ~]$ sudo mkdir -p /data/input /data/output
[ons@mylocalhost ~]$
```

🔍 Explication :

- `-p` : crée les dossiers et leurs parents si nécessaire.
- Ces deux dossiers vont servir à **échanger des fichiers** entre l'hôte et le conteneur (comme des dossiers partagés).

☑ 3. Changer le propriétaire et les droits d'accès

```
sudo chown -R pod:pod /data/input /data/output
sudo chmod -R 777 /data/input /data/output
```

```
[ons@mylocalhost ~]$ sudo chown -R pod:pod /data/input /data/output
[ons@mylocalhost ~]$ sudo chmod -R 777 /data/input /data/output
[ons@mylocalhost ~]$
```

🔍 Explication :

- `chown -R pod:pod` : donne la **propriété totale** à l'utilisateur `pod`.
- `chmod -R 777` : tous les utilisateurs auront tous les droits (lecture, écriture, exécution).
  ⚠ Attention : ce niveau de permission est à usage pédagogique, à éviter en production.

☑ 4. Se connecter en SSH sur le compte `pod`

```
ssh pod@localhost
```

```
[ons@mylocalhost ~]$ ssh pod@localhost
pod@localhost's password:
Register this system with Red Hat Insights: rhc connect

Example:
# rhc connect --activation-key <key> --organization <org>

The rhc client and Red Hat Insights will enable analytics and additional
management capabilities on your system.
View your connected systems at https://console.redhat.com/insights

You can learn more about how to register your system
using rhc at https://red.ht/registration
[pod@mylocalhost ~]$
```

🔍 **Explication :**

- On se connecte au compte `pod` en local, pour réaliser les prochaines actions **en tant qu'utilisateur non root**.

☑ 5. Créer 5 fichiers `.txt` contenant un message personnalisé

```
for i in {1..5}; do echo "Hello $USER, test file $i" > /data/input/file$i.txt; done
```

```
[pod@mylocalhost ~]$ for i in {1..5}; do echo "Hello $USER, test file $i" > /data
/input/file$i.txt; done
[pod@mylocalhost ~]$
```

🔍 **Explication :**

- Cette boucle crée 5 fichiers dans `/data/input`.
- Chaque fichier contient un texte personnalisé, utile pour **tester la conversion en PDF**.

☑ 6. Télécharger les fichiers nécessaires pour construire l'image

```
wget https://raw.githubusercontent.com/sachinyadav3496/Text-To-
PDF/master/pdf_converter.py
wget https://raw.githubusercontent.com/sachinyadav3496/Text-To-PDF/master/Dockerfile
```

```
33|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1991 (1.9K) [text/plain]
Saving to: ‘pdf_converter.py’

pdf_converter.py     100%[===================>]   1.94K  --.-KB/s    in 0s

2026-05-04 11:26:11 (26.2 MB/s) - ‘pdf_converter.py’ saved [1991/1991]

--2026-05-04 11:26:11--  https://raw.githubusercontent.com/sachinyadav3496/Text-T
o-PDF/master/Dockerfile
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.13
3, 185.199.109.133, 185.199.108.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.1
33|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 163 [text/plain]
Saving to: ‘Dockerfile’

Dockerfile           100%[===================>]     163  --.-KB/s    in 0s

2026-05-04 11:26:11 (8.70 MB/s) - ‘Dockerfile’ saved [163/163]
```

🔍 **Explication :**

- `wget` permet de télécharger des fichiers en ligne de commande.
- Ces fichiers vont nous servir à **construire une image personnalisée**, contenant un script Python de conversion texte ➜ PDF.

☑ 7. Construire l’image `pdf`

```
podman build -t pdf .
ping and want to suppress this warning.
--> 20a29e561c0f
STEP 4/7: RUN mkdir /temp
--> 8ef7e7345d21
STEP 5/7: COPY pdf_converter.py /app/
--> 7d5d35b31c6a
STEP 6/7: WORKDIR /app
--> fff9820fc1c9
STEP 7/7: CMD ["python3", "pdf_converter.py"]
COMMIT pdf
--> 857bf52d7d65
Successfully tagged localhost/pdf:latest
857bf52d7d654c92b246538619e7cd81ae6859959cfeaf8fc02ede7bb74fc0de
```

🔍 Explication :

- `build` : construit une image à partir d'un `Dockerfile` présent dans le répertoire courant (`.`).
- `-t pdf` : donne le nom `pdf` à l'image créée.

📝 Résultat attendu :

- L'image nommée `localhost/pdf` est générée avec succès.

☑ 8. Vérifier l'image créée

```
podman images
```

```
[pod@mylocalhost ~]$ podman images
REPOSITORY                          TAG         IMAGE ID      CREATED         SIZE
localhost/pdf                       latest      857bf52d7d65  36 seconds ago  345
MB
registry.fedoraproject.org/fedora   latest      113f8751d76f  4 hours ago     189
MB
[pod@mylocalhost ~]$
```

🔍 Explication :

- Affiche toutes les images disponibles localement.
- On doit y voir l'image `pdf` avec sa taille et date de création.

☑ 9. Exécuter le conteneur avec montage des répertoires

```
podman run --name pdfconverter -v /data/input:/data/input -v /data/output:/data/output
pdf
```

```
[pod@mylocalhost ~]$ podman run --name pdfconverter \
  -v /data/input:/data/input:Z \
  -v /data/output:/data/output:Z \
  pdf
```

🔍 Explication :

- `--name pdfconverter` : nomme le conteneur.
- `-v /data/input:/data/input` : **monte le dossier local `/data/input` dans le conteneur**, au même chemin.
- `-v /data/output:/data/output` : idem pour `/data/output`.
- `pdf` : nom de l'image à utiliser.

Cela permet au script dans le conteneur d'accéder aux fichiers `.txt` et de générer les fichiers PDF dans `/data/output`.

☑ 10. Tester le fonctionnement du conteneur

Vérifie le dossier `/data/output` :

```
ls /data/output
```

```
[pod@mylocalhost ~]$ ls /data/output
file1.pdf  file2.pdf  file3.pdf  file4.pdf  file5.pdf
[pod@mylocalhost ~]$
```

Explication :

- Tu devrais voir apparaître les fichiers PDF (file1.pdf, file2.pdf, etc.)
- Cela confirme que le conteneur a bien exécuté le script de conversion.

☑ 11. Générer un service systemd pour le conteneur

```
podman generate systemd --name pdfconverter --files --restart-policy=always
```

```
[pod@mylocalhost ~]$ podman generate systemd --name pdfconverter --files --restar
t-policy=always

DEPRECATED command:
It is recommended to use Quadlets for running containers and pods under systemd.

Please refer to podman-systemd.unit(5) for details.
/home/pod/container-pdfconverter.service
[pod@mylocalhost ~]$
```

Explication :

- Crée un fichier `container-pdfconverter.service` qui permet de gérer le conteneur comme un service Linux.
- `--restart-policy=always` : assure le redémarrage automatique du conteneur si besoin.

☑ 12. Activer le service systemd au démarrage

```
mkdir -p ~/.config/systemd/user
cp container-pdfconverter.service ~/.config/systemd/user/

systemctl --user daemon-reexec
```

```
systemctl --user daemon-reload
systemctl --user enable container-pdfconverter.service
systemctl --user start container-pdfconverter.service
```

```
[pod@mylocalhost ~]$ mkdir -p ~/.config/systemd/user
[pod@mylocalhost ~]$ cp container-pdfconverter.service ~/.config/systemd/user/
[pod@mylocalhost ~]$ systemctl --user daemon-reexec
[pod@mylocalhost ~]$ systemctl --user daemon-reload
[pod@mylocalhost ~]$ systemctl --user enable container-pdfconverter.service
Created symlink /home/pod/.config/systemd/user/default.target.wants/container-pdfconv
erter.service → /home/pod/.config/systemd/user/container-pdfconverter.service.
[pod@mylocalhost ~]$ systemctl --user start container-pdfconverter.service
[pod@mylocalhost ~]$
```

🔍 Explication :

- Le service est maintenant **activé et lancé automatiquement** à chaque démarrage de session de l'utilisateur `pod`.
